3.2.2.3 Algorithm

The system algorithm defines the computational workflow of the two-way communication software between a deaf-mute individual and a non-signer, and is presented in Algorithms 1 to 6. Algorithm 1 describes the main program, consisting of the initialization of the system components and a continuous loop that routes every conversational turn to the appropriate pipeline. Algorithms 2 and 3 describe the two pipelines, Case 1 (signer to non-signer) and Case 2 (non-signer to signer), each organized into blocks that follow the stages of the system pipeline. Algorithms 4 to 6 define the procedures shared by these pipelines: Space Restriction, Active Context, and Sentence Completion.

During initialization, the system loads the vocabulary of 34 confirmed signs and a lookup table of 91 words, composed of the signs and the helper words that occur in natural speech. A word-level tokenizer assigns each word a unique identifier, and the two trained networks are loaded from their saved parameters: the word picker, an encoder used in Case 2, and TinyLlama, a decoder used in Case 1. Both networks are set to evaluation mode so that identical inputs always produce identical outputs. The system then verifies that the microphone and the offline speech-to-text model are available and initializes an empty context memory, which holds the person being discussed, the last question, and the active conversation topic. The seven topics and their trigger signs were derived from interviews with deaf students (help and repair, school and learning, whereabouts and plans, greeting, courtesy, home and family, and feelings). Once this startup sequence concludes, the main loop waits for a mode selection, executes the corresponding pipeline, and displays the resulting text on the screen facing the intended reader.

Algorithm 1: Two-Way Sign-Speech Communication System (Main Program)

Input: mode selection; signs with confidence (Case 1) or speech / typed text (Case 2)

Output: context-aware text on the display facing the intended reader

Initialize:
    SIGN VOCABULARY ← 34 confirmed signs;
    LOOKUP TABLE ← SIGN VOCABULARY ∪ HELPER WORDS;
    tokenizer ← buildTokenizer(LOOKUP TABLE); // 91 words, 95 token IDs
    wordPicker ← loadModel(trained word picker); // encoder, Case 2
    tinyLlama ← loadModel(trained TinyLlama); // decoder, Case 1
    set *wordPicker* and *tinyLlama* to evaluation mode;
    micReady ← isAvailable(microphone, speech-to-text model);
    context memory ← empty; // person, questions, asked to you, topic
    TOPICS ← 7 topics with their trigger signs; // from the interviews
    Define MAX SEQUENCE LENGTH ← 16;
    Define MAX OUTPUT LENGTH ← 16;
    Define SIGN CONFIDENCE THRESHOLD ← 0.6;
    Define SAMPLE RATE ← 16,000; // the speech model's sample rate
    Define MAX RECORD SECONDS ← 8, NO SPEECH SECONDS ← 5;

while system is active **do**
    mode ← SELECT MODE(); // option 1 → CASE 1; options 2–3 → CASE 2
    if *mode* == CASE 1 (Signer → Non-signer) **then**
        signs ← receiveSigns(); // each sign with its confidence
        // Algorithm 2
        refined text ← SIGN TO SENTENCE(*signs*, context memory);
        display(*refined text*, screen facing the non-signer);
    else if *mode* == CASE 2 (Non-signer → Signer) **then**
        typed ← typed sentence, or none when using the microphone;
        // Algorithm 3
        refined text ← SPEECH TO GLOSS(*typed*, context memory);
        display(*refined text*, screen facing the deaf-mute individual);
    else if *mode* == RESET **then**
        context memory ← empty;
    else if *mode* == QUIT **then**
        break;
    end if

34 **end while**

In Case 1, the sign words recognized by the gesture recognition module are received together with the recognizer's confidence. A sign whose confidence is below the threshold is discarded, because the interviewed students mix gesture and home sign into their signing and the system must not guess. A negation (NOT) is kept as a flag rather than discarded, so that the meaning of the sentence is never inverted. The remaining signs are normalized into sign names, tokenized, and embedded by TinyLlama's sign embedding table without positional information, since the meaning of a sign sequence depends on which signs were produced rather than on the order in which the recognizer delivered them. The Active Context block updates the conversation topic and applies the repair rule: when NOT and UNDERSTAND are signed together, the system asks the hearing person to repeat or write the message. Otherwise it supplies a missing person from the context memory, but only when the signs are about a person. The Sentence Completion block then generates the English sentence one word at a time, rewrites a reply to a question as a statement, inserts the negation when NOT was signed, and replaces any sentence that omits a signed word with the signs themselves. Finally, the sentence is shown on the display facing the non-signer with a question mark when it asks something and a full stop otherwise; punctuation is added only for the hearing reader.

Algorithm 2: Sign-to-Sentence Pipeline (Case 1: Signer → Non-signer)

Input: *signs* (recognized sign words with their confidence), context memory
Output: *refined text* (English sentence for the display facing the non-signer)

```
1    // BLOCK 1 --- TRANSCRIBED WORD
2    kept ← [ ]; negated ← false;
3    for each (token, confidence) in signs do
4        if confidence < SIGN CONFIDENCE THRESHOLD then
```

```
        skip token; // unsure sign: never guessed
    else if token ∈ NEGATIONS then
        negated ← true; // NOT is kept as a flag, never dropped
    else if normalizeWord(token) ∈ SIGN VOCABULARY then
        append normalizeWord(token) to kept; // e.g., mama → mother
    end if
end for
signs ← first MAX SEQUENCE LENGTH words of kept;
if signs is empty AND negated then
    return "no"; // only NOT was signed
else if signs is empty then
    return "";
end if
// BLOCK 2 --- TOKENIZE
ids ← tokenizer.encode(signs);
// BLOCK 3 --- EMBEDDING
M ← tinyLlama.embedSource(ids); // one 64-value vector per sign, no position
// BLOCK 4 --- ACTIVE CONTEXT (Algorithm 5)
repair ← negated AND "understand" ∈ signs;
UPDATE TOPIC(signs, repair, context memory); // NOT counts if negated
if NOT repair then
    (signs, person) ← FILL SUBJECT(signs, context memory);
    if person ≠ none then
        M ← concatenate(tinyLlama.embedSource(person), M);
    end if
end if
// BLOCK 5 --- SENTENCE COMPLETION (Algorithm 6)
if repair then
    refined text ← REPAIR SENTENCE; // ask to say it again or write it
else
    refined text ← GENERATE SENTENCE(M);
    if context memory.last was question then
        refined text ← toDeclarative(refined text); // question form → statement
    end if
    if some sign in signs is missing from refined text then
        refined text ← join(signs); // safety net: show the signs
```

```
        if negated then refined text ← "not " + refined text;
    else if negated then
        refined text ← NEGATE(refined text); // i am fine → i am not fine
    end if
end if
NOTE DEAF(signs, refined text, context memory);
// BLOCK 6 --- DISPLAY
return ADD PUNCTUATION(refined text); // Algorithm 6
```

In Case 2, the voice of the non-signer is recorded at 16,000 samples per second, the sample rate of the speech-to-text model, and the recording stops as soon as the speaker stops talking. The audio is read in quarter-second chunks and passed to Vosk, a pre-trained speech-to-text model that runs offline, whose built-in end-of-speech rules end the recording after 0.5 to 1.0 seconds of silence following speech (0.88 seconds on average in our measurements). If nobody speaks within five seconds, matching the model's own no-speech rule, the recording is abandoned, and no turn may exceed eight seconds, enough for the sixteen words the models accept. When a sentence is typed instead, it is used directly as the transcript. The system first notes whether the hearing person asked a question, including a yes/no question, so that the reply can be written as a statement. The transcript is normalized by dropping greeting phrases outside the vocabulary, such as good morning, removing punctuation, expanding contractions, and replacing untrained synonyms with the words the model learned, and is then reduced to the words contained in the lookup table; a negation is removed together with the word it negates when that word is outside the lookup table, so that do not worry cannot reach the reader as a bare not. The reduced words are tokenized and embedded with both word and position information, and two transformer blocks contextualize every word against the whole sentence. The Space Restriction block then decides which words are kept, so that only words with a corresponding sign remain,

in the order in which they were spoken. Finally, the Active Context block resolves pronouns to the remembered person and updates the conversation topic before the glosses are displayed facing the deaf-mute individual as plain sign words, without punctuation.

Algorithm 3: Speech-to-Gloss Pipeline (Case 2: Non-signer → Signer)

Input: *typed* (typed sentence, or none to use the microphone), context memory
Output: *refined text* (sign-word glosses for the display facing the deaf-mute individual)

```
1   if typed = none then
2       // BLOCK 1 --- VOICE RECORDING (stops when the speaker stops)
3       audio ← [ ]; heard ← false;
4       repeat
5           append readMicrophone(0.25 s, SAMPLE RATE) to audio;
6           if vosk.endOfSpeech(audio) AND heard then break; // ≈0.9 s silence
7           if vosk.hearsWords(audio) then heard ← true;
8           if MAX RECORD SECONDS reached then break;
9           if no words heard after NO SPEECH SECONDS then break;
10      end repeat
11      // BLOCK 2 --- SPEECH TO TEXT
12      transcript ← vosk.transcribe(audio); // pre-trained, offline
13  else
14      transcript ← typed; // typed input skips Blocks 1–2
15  end if
16  // BLOCK 3 --- NORMALIZATION
17  asked ← transcript has "?" OR starts with a QUESTION STARTER;
18  text ← lowercase(transcript) without GREETING PHRASES; // good morning
19  text ← removePunctuation(text);
20  words ← [ ];
21  for each w in split(text) do
22      if w ∈ CONTRACTIONS then
23          w ← expand(w); // don't → do not
24      else if w is a possessive of a LOOKUP TABLE word then
25          w ← baseWord(w); // dad's → father
26      else if w ∈ SYNONYMS AND w ∉ TRAINED WORDS then
27          w ← SYNONYMS[w]; // mom → mother
```

```
        end if
        append w to words;
    end for
    // BLOCK 4 --- REDUCTION
    kept ← [ ];
    for each w in words do
        if w ∈ NEGATIONS AND it negates a word ∉ LOOKUP TABLE then
            skip w; // "do not worry": the not goes with worry
        else if w ∈ LOOKUP TABLE then
            append w to kept;
        end if
    end for
    kept ← first MAX SEQUENCE LENGTH words of kept;
    if kept is empty then
        return "";
    end if
    // BLOCK 5 --- TOKENIZE
    ids ← tokenizer.encode(kept);
    // BLOCK 6 --- EMBEDDING
    X ← wordPicker.embed(ids); // word vector + position vector, n × 64
    // BLOCK 7 --- CONTEXTUALIZE
    for layer ← 1 to 2 do
        X ← TransformerBlock(layer, X); // self-attention + feed-forward
    end for
    // BLOCK 8 --- SPACE RESTRICTION (Algorithm 4)
    (tagScores, questionScores) ← wordPicker.heads(X);
    glosses ← RESTRICT TO SIGNS(tagScores, questionScores, kept);
    // BLOCK 9 --- ACTIVE CONTEXT (Algorithm 5)
    glosses ← RESOLVE PRONOUN(words, glosses, context memory);
    NOTE HEARING(words, glosses, asked, context memory);
    repair ← some w ∈ NEGATIONS AND "understand" ∈ words;
    UPDATE TOPIC(words ∪ glosses, repair, context memory);
    // BLOCK 10 --- TRANSCRIBED WORDS
    refined text ← join(glosses); // plain sign words, no punctuation
    // BLOCK 11 --- DISPLAY
    return refined text; // shown on the display facing the deaf-mute individual
```

The Space Restriction block converts the scores of the word picker into the glosses shown to the deaf-mute individual. For every word, the tag head scores whether the word is dropped or kept, and the question head decides whether the marker “how” must precede the output. A word without a corresponding sign can never be kept, and every sign that carries the message is always kept; only “my” and “you”, the two signs that the training data itself leaves out of spoken sentences, are left to the picker's keep-or-drop decision. The kept signs appear in the order in which they were spoken, which is the order of 604 of the 608 training targets, because the picker's position scores scrambled sentences it had not seen. A spoken negation is shown as “not” where it was said, so that the meaning is not inverted; “fine” is absorbed by the marker “how” in a question about someone's wellbeing, the object pronoun “me” is not shown as the sign MY, and the same sign is never shown twice in a row.

Algorithm 4: Space Restriction (RESTRICT TO SIGNS)

Input: *tagScores* (n × 17), *questionScores* (2), words $w_1 \ldots w_n$
Output: *glosses* (sign words in signing order)

Define OPTIONAL SIGNS ← {my, you}; // the only signs the data leaves out
isQuestion ← *questionScores*[1] > *questionScores*[0];
glosses ← [ ]; *negated* ← false;
for $i \leftarrow 1$ to n **do** // in the order the words were spoken
    $s \leftarrow$ normalizeWord(w_i);
    if $w_i \in$ NEGATIONS **then**
        if NOT *negated* **then** append “not” to *glosses*; // where it was said
        negated ← true;
    else if $s \notin$ SIGN VOCABULARY OR w_i = “me” **then**
        skip w_i; // no sign to show; “eat with me” is not MY
    else if s = “fine” AND *isQuestion* **then**
        skip w_i; // “is mother okay” → how mother
    else if $s \in$ OPTIONAL SIGNS AND

```
            tagScores[i, 0] ≥ max(tagScores[i, 1…16]) then // drop score wins
        skip w_i; // only my / you may be dropped by the picker
    else if glosses is empty OR last of glosses ≠ s then
        append s to glosses; // never the same sign twice in a row
    end if
end for
if isQuestion then
    glosses ← ["how"] + glosses;
end if
return glosses;
```

The Active Context block maintains a structured conversation memory shared by both cases: the last person mentioned, the question words of the last turn, whether the last question was addressed to the deaf-mute individual, and the active conversation topic. In Case 2, it replaces the pronouns “he” and “she” with the remembered person when the person fits the pronoun, and it records whether the hearing person asked a question, including a yes/no question such as “is your mother here?”, so that the Deaf reply is written as a statement. In Case 1, it supplies the remembered person only when the signs are about a person, such as a single place sign answering a question about where someone is; a question about a thing, a time, or an activity is never given a person, following the interviewed students' request that the system output only what was signed. The topic follows the seven contexts found in the interviews. One topic is active at a time, but every context keeps a weight between 0 and 1. Each turn, the share of every context is measured over the turn's context words, a word that fits the active context being read in that context, and every weight moves halfway toward its share. Signs from another context therefore leak into the conversation without switching it, and a context that is no longer mentioned fades gradually instead of dropping to zero. The topic changes only when another context

covers at least half of the turn's context words and more than the active one, for example 9 of 16; ties are resolved by the weights carried from earlier turns, and NOT with UNDERSTAND always switches to help and repair.

Algorithm 5: Active Context (shared conversation memory)

Input: context memory, the words or signs of the current turn
Output: updated glosses or signs, updated context memory

```
1   procedure RESOLVE PRONOUN(words, glosses, memory)
2       if memory.person ≠ none AND no gloss ∈ PEOPLE then
3           if ("he" ∈ words AND memory.person ∈ {father, teacher, friend}) OR
4              ("she" ∈ words AND memory.person ∈ {mother, teacher, friend}) then
5               insert memory.person after the leading question words;
6           end if
7       end if
8       return glosses;
9   end procedure
10  procedure NOTE HEARING(words, glosses, asked, memory)
11      memory.question words ← QUESTION WORDS in words ∪ glosses;
12      memory.last was question ← (memory.question words ≠ ∅) OR asked;
13      if "you" ∈ words then
14          memory.asked to you ← true;
15      else if some word ∈ PEOPLE then
16          memory.asked to you ← false;
17      end if
18      memory.person ← the last word ∈ PEOPLE (unchanged when none);
19  end procedure
20  procedure FILL SUBJECT(signs, memory)
21      if some sign ∈ PEOPLE OR "my" ∈ signs OR "you" ∈ signs OR
22         memory.person = none then
23          return (signs, none); // already says who, or nobody remembered
24      end if
25      aboutPerson ← (some sign ∈ QUESTION WORDS AND
26          every other sign ∈ STATES) OR
27          (some sign ∈ STATES AND no sign ∈ ACTIVITIES) OR
```

```
28          ("where" ∈ memory.question words AND every sign ∈ PLACES);
29      if aboutPerson AND NOT memory.asked to you then
30          return ([memory.person] + signs, memory.person);
31      end if
32      return (signs, none); // e.g. "where water" is about the water
33  end procedure
34  procedure NOTE DEAF(signs, sentence, memory)
35      memory.last was question ← false; memory.question words ← ∅;
36      if some word of signs or sentence ∈ PEOPLE then
37          memory.asked to you ← false;
38          memory.person ← the last such word;
39      end if
40  end procedure
41  procedure UPDATE TOPIC(words, repair, memory)
42      Define SWITCH SHARE ← 0.5, FADE ← 0.5;
43      C ← the signs of words that trigger some topic; // context words
44      if C is empty then
45          return; // nothing to measure: topic and weights stay
46      end if
47      T ← memory.topic; W ← memory.weights;
48      for each topic in TOPICS do
49          share[topic] ← |topic signs in C| / |C|; // weighted by confidence
50          W[topic] ← W[topic] + FADE × (share[topic] − W[topic]);
51      end for // a sign of T counts only for T; the others leak in
52      if repair then
53          T ← "help and repair"; // NOT + UNDERSTAND
54      else if T = none then
55          T ← the topic with the highest share;
56      else
57          best ← the other topic with the highest share; // tie: higher W
58          if share[best] ≥ SWITCH SHARE AND share[best] > share[T] then
59              T ← best; // a whole-context change, e.g., 9 of 16
60          end if
61      end if
62      memory.topic ← T;
63  end procedure
```

The Sentence Completion block generates the English sentence with TinyLlama one word at a time. At every step, the words generated so far pass through two decoder blocks, in which causal self-attention with rotary position embeddings relates each word only to the words before it, cross-attention reads the embedded signs, and a SwiGLU feed-forward layer refines the result. The output head then scores every word of the vocabulary, and the highest-scoring word is appended (greedy decoding) until the end-of-sentence token is produced. When NOT was signed, the NEGATE procedure inserts the negation after the sentence's helper verb, or adds “do not” when the sentence has none, so that a signed negation always reaches the hearing person. Last, the ADD PUNCTUATION procedure ends the sentence with a question mark when it asks something, that is, when it starts with a question starter, contains a question word, or contains a question starter followed by a subject, and with a full stop otherwise. Punctuation is added only for the hearing reader; the deaf-mute individual's display shows plain sign words.

Algorithm 6: Sentence Completion (GENERATE SENTENCE, NEGATE, ADD PUNCTUATION)

Input: *M* (embedded signs, k × 64); a sentence to negate or to punctuate
Output: *sentence* (English sentence)

```
1   procedure GENERATE SENTENCE(M)
2       y ← [<bos>]; // start-of-sentence token
3       for t ← 1 to MAX OUTPUT LENGTH do
4           H ← tinyLlama.embedToken(y);
5           for layer ← 1 to 2 do
6               H ← H + CausalSelfAttention(RMSNorm(H)); // earlier words only
7               H ← H + CrossAttention(RMSNorm(H), M); // reads the signs
8               H ← H + SwiGLU(RMSNorm(H));
9           end for
10          scores ← OutputHead(RMSNorm(H[last])); // 95 word scores
11          next ← argmax(scores); // greedy decoding
```

```
            if next = <eos> then
                break;
            end if
            append next to y;
        end for
        return tokenizer.decode(y without <bos>);
    end procedure
    procedure NEGATE(sentence) // a signed NOT is never lost
        words ← split(toDeclarative(sentence));
        if first word ∈ HELPER VERBS then // still a question: are you studying
            insert “not” after the second word;
        else if some word ∈ HELPER VERBS then // i am fine → i am not fine
            insert “not” after the first helper verb;
        else if first word ∈ {i, you, we} then // i want water → i do not want water
            insert “do not” after the first word;
        else
            put “do not” in front; // help me → do not help me
        end if
        return join(words);
    end procedure
    procedure ADD PUNCTUATION(sentence) // the hearing reader's display only
        if first word ∈ QUESTION STARTERS OR
           some word ∈ QUESTION WORDS OR
           a QUESTION STARTER is followed by you, i, he, she, it, we, they then
            return sentence + “?”; // where is the water? / can you say it again?
        end if
        return sentence + “.”; // mother is home.
    end procedure
```

For the group only (not for the manuscript): where each block is in the code

Algorithm 1, Initialize	full_demo.py: main() → slm_pipeline.py: SLMPipeline.__init__(), _load(), TOPICS, SIGN_CONFIDENCE_THRESHOLD; tokenizer.py; speech_input.py: is_available(), SAMPLE_RATE, MAX_RECORD_SECONDS, NO_SPEECH_SECONDS
Algorithm 1, main loop	full_demo.py: main() menu loop (1, 2, 3, r, q), screen(); slm_pipeline.py: deaf_to_non_deaf(), non_deaf_to_deaf()
Algorithm 2, Block 1	slm_pipeline.py: transcribed_signs() (confidence filter, NEGATIONS); make_labels.py: normalize_word()
Algorithm 2, Blocks 2–3	slm_pipeline.py: tokenize(), embedding(ids, "signs") → decoder_tinyllama.py: embed_source()
Algorithm 2, Block 4	slm_pipeline.py: deaf_to_non_deaf() repair check; ActiveContext.update_topic(), fill_subject(); torch.cat()
Algorithm 2, Block 5	slm_pipeline.py: REPAIR_SENTENCE, sentence_completion(), to_declarative(), covered(), negate()
Algorithm 2, Block 6	slm_pipeline.py: deaf_to_non_deaf() display(punctuate(...)); punctuate(), QUESTION_STARTERS, SUBJECTS
Algorithm 3, Blocks 1–2	slm_pipeline.py: voice_recording(), speech_to_text() → speech_input.py: record_until_silence(), capture(), transcribe() (Vosk)
Algorithm 3, Block 3	slm_pipeline.py: asked_question(), normalization(), GREETING_PHRASES, CONTRACTIONS, known_form(); onnx_config.py: GLOSS_MAP
Algorithm 3, Block 4	slm_pipeline.py: reduction(), negated_word(), NEGATION_SKIP, LOOKUP_TABLE
Algorithm 3, Blocks 5–7	slm_pipeline.py: tokenize(), embedding(), contextualize() → word_picker.py: embed(), contextualize()
Algorithm 3, Block 8 / Algorithm 4	slm_pipeline.py: space_restriction() → word_picker.py: heads(), restrict_to_signs(), OPTIONAL_SIGNS
Algorithm 3, Block 9 / Algorithm 5	slm_pipeline.py: ActiveContext.resolve_pronoun(), note_hearing(asked), fill_subject(), note_deaf(), update_topic()
Algorithm 6	decoder_tinyllama.py: generate_from_memory(), decode(), TinyLlamaBlock; slm_pipeline.py: negate(), punctuate()